

Chapter 3: Some Simple Numeric Programs

EEC 3150

Trevecca Nazarene University

Outline

- 1 Exhaustive Enumeration (Brute Force)
- 2 The Bisection Method
- 3 Floating-Point Arithmetic
- 4 The Newton-Raphson Method

Exhaustive Enumeration (Brute Force)

Exhaustive Enumeration (Brute Force)

Exhaustive enumeration (brute force):

Exhaustive enumeration (more commonly **brute force**) is an approach to problem-solving which naively tests every possible solution to a problem

Pros:

- Easy to implement and understand
- For simple problems, computers are fast enough

Cons:

- For some problems, the space of possible solutions is enormous and exploring all of them is intractable for even a billion supercomputers working together

For which problem is brute force a better fit?

- Guessing someone's 4-digit PIN
- Finding the maximum value of a real-valued function of 100 variables

Exhaustive Enumeration (Brute Force)

Code example:

Find the cube root of a perfect cube

```
x = 64
ans = 0

while ans**3 < abs(x):
    ans = ans + 1
#end

if ans**3 != abs(x):
    print(x, 'is not a perfect cube')
else:
    if x < 0:
        ans = -ans
    #end
    print('Cube root of', x, 'is', ans)
#end
```

- Common coding strategy: start with an initial guess at a solution (ans=0) then use a loop to iterate through other possible solutions
- Code tests ALL possible solutions until it either finds the solution or has eliminated all possible solutions
- How to know all possible solutions have been eliminated?
- While loop stopping condition?

Exhaustive Enumeration (Brute Force)

Code example:

Guess someone's 4-digit PIN (0-9) by listing all possible PINs

```
for a in range(10):
    for b in range(10):
        for c in range(10):
            for d in range(10):
                print( str(a)+str(b)+
                      str(c)+str(d) )
```

- List all 10 possible choices for each of the 4 digits
- Use `range(10)` to generate integers
- Use `str()` to take advantage of string concatenation (+)
- Best case: first guess
- Worst case: last guess (10,000)
- Average case: $10,000/2 = 5,000$

Approximate Solutions and Brute Force

Approximate solutions:

Problem: find the square root of any non-negative number

Issue with the problem statement:

- Square root of 2 is irrational, requires infinite digits
- Computers have a finite level of precision (number of digits) and could only give an approximate value

Better problem statement:

- Approximate the square root of any non-negative number with a precision of ϵ

Approximate Solutions and Brute Force

Example:

Approximate the square root of x to within a precision of ϵ

```
x = 100
```

```
eps = 0.01
```

```
step = 0.001
```

```
ans = 0.0
```

```
while abs(ans**2 - x) >= eps and ans**2 <= x:  
    ans += step  
#end
```

```
if abs(ans**2 - x) >= eps:  
    print('Failed on square root of', x)  
else:  
    print('sqrt(', x, ') is approx.', ans)  
#end
```

- Initial guess `ans = 0.0`
- Checks whether `ans` is a solution or is beyond the scope of possible solutions (while loop condition)
- Until the while loop is done, it increments `ans` by `step` at each iteration

Question: What happens if the step size is too big?

Approximate Solutions and Brute Force

Example:

Approximate the square root of x to within a precision of ϵ

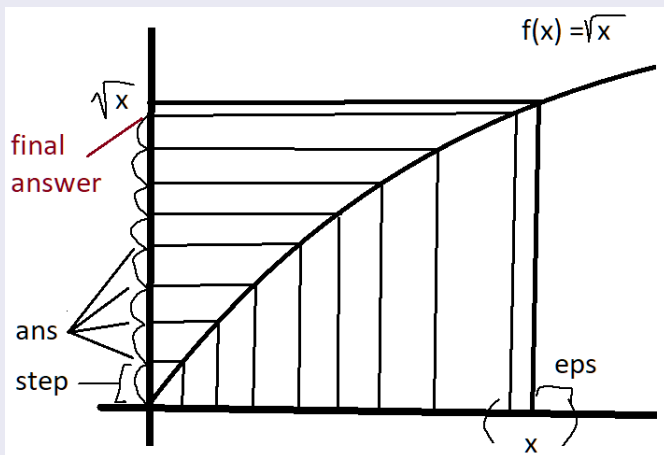


Figure: Visual depiction of the algorithm

The Bisection Method

Approximate Solutions and the Bisection Method

Motivating the bisection method:

We almost always want better algorithms than naive brute force approaches

Consider the problem of looking up *median* in the dictionary:

- The dumb way (brute force): start at the beginning and look at every word until we find it
- The smart way:
 1. *The words are all sorted alphabetically*
 2. Guess a page that you think is close
 3. If the page contains words before *median*, you need to flip further ahead (or vice versa)
 4. Keep guessing closer and closer pages until you arrive at the correct page

Approximate Solutions and the Bisection Method

Motivating the bisection method:

Look up *median* in the dictionary:

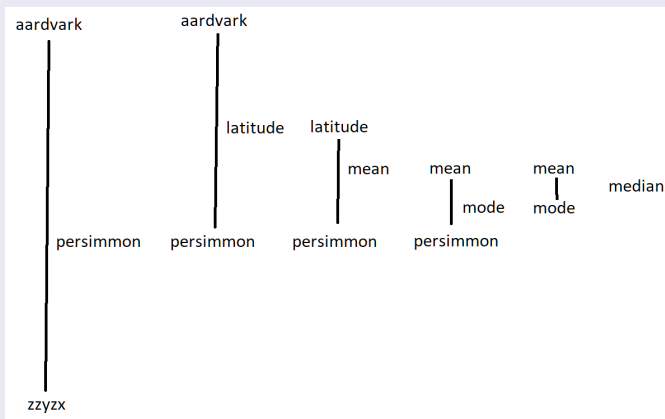


Figure: Visual depiction of the algorithm

Approximate Solutions and the Bisection Method

Example:

Approximate the square root of y to within a precision of ϵ

Solve $f(x) = x^2 - y = 0$,
where $y = 10$

```
y = 10
```

```
eps = 0.01
```

```
low = 0.0
```

```
high = max(1.0, y)
```

```
ans = (high + low)/2
```

```
while abs(ans**2 - y) >= eps:
```

```
    if ans**2 < y:
```

```
        low = ans
```

```
    else:
```

```
        high = ans
```

```
    #end
```

```
    ans = (high + low)/2
```

```
#end
```

```
print('sqrt(',y,') is approx.', ans)
```

- Make sure that the real answer is between your initial low, high guesses
- At each step, your new ans guess should be somewhere in between low,high (the mean is the easiest solution)
- Re-assign the value of low or high depending on whether ans was above or below the real answer

Approximate Solutions and the Bisection Method

Motivating the bisection method:

Approximate $\sqrt{10}$ with the bisection method:

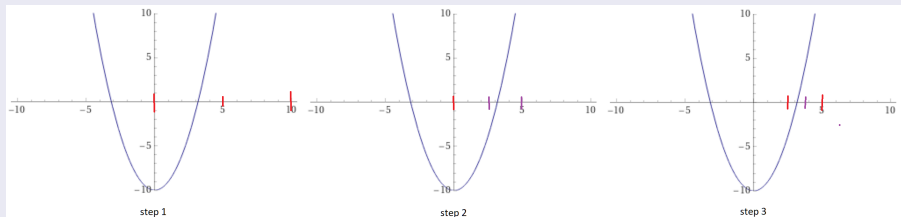


Figure: Visual depiction of the algorithm

Floating-Point Arithmetic

Approximate Solutions and the Bisection Method

Question:

What do you expect the follow code to print?

```
total = 0.0
for i in range(15):
    total += 0.1
    print(total)
```

Floating-Point Arithmetic

Example:

We get

```
0
0.1
0.2
0.30000000000000004
0.4
0.5
0.6
0.7
0.7999999999999999
0.8999999999999999
0.9999999999999999
1.0999999999999999
1.2
1.3
1.4000000000000001
1.5000000000000002
```

Why?

- In computers, numbers are represented as a binary format instead of decimal format
- Computers only have a finite level of precision for numbers (i.e., number of digits)
- 0.1 cannot be represented exactly with a finite number of digits in binary
- Repeatedly adding it accumulates errors

Why can't 0.1 be represented exactly in binary???

Floating-Point Arithmetic

Number representation in decimal format:

In decimal format, numbers are represented as a sum of multiples of powers of 10 (the base of the decimal system)

$$\begin{aligned}123.4 &= 100 + 20 + 3 + 0.4 \\&= 1 \cdot 100 + 2 \cdot 10 + 3 \cdot 1 + 4 \cdot 0.1 \\&= 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 + 4 \cdot 10^{-1} \\&= \sum_{i=-1}^2 d_i \cdot 10^i\end{aligned}$$

where $d_{-1} = 4, d_0 = 3, d_1 = 2, d_2 = 1$

Also, in scientific notation, $123.4 = 1.234 \cdot 10^{-2}$

Floating-Point Arithmetic

Number representation in decimal format:

Inspired by scientific notation, we could choose to represent this number as just a list of digits and an exponent (this is how floating-point systems work):

This assumes that we know that the base of this system (in the decimal case, 10)

- All digits must be integers between 0-9 (can't be 10 or greater)
- Use powers of 10 to reconstruct the number

$$(1234, 2) \rightarrow 1.234 \cdot 10^2 = 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 + 4 \cdot 10^{-1} = 123.4$$

(NOTE: I'm using the convention of Section 1.3 in Heath, not 3.3 in Guttag)

Floating-Point Arithmetic

Number representation in binary format:

We can use whatever base we want (base 2 is binary)

- All digits must be integers 0, 1
- Use powers of 2 to reconstruct the number

$$\begin{aligned}(10011, 2) \rightarrow 1.0011 \cdot 2^2 &= 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 100.11_2 \\ &= 1 \cdot 4 + 0 \cdot 2 + 0 \cdot 1 + 1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} = 4.75_{10}\end{aligned}$$

Conversion from binary to decimal: $100.11_2 =$

Floating-Point Arithmetic

Defm: floating-point system:

Define the following integers:

- b : the base (use powers of this number)
- p : the precision (how many digits)
- L, U : the lower and upper limits of the exponent

Floating-point numbers have two parts:

- The *mantissa*: $d_0 d_1 d_2 \cdots d_{p-1}$
- The *exponent*: E

Then, any **floating-point number** $x \in \mathbb{F}$ has the form

$$x = \pm b^E \cdot \sum_{i=0}^{p-1} \frac{d_i}{b^i}$$

where all the d_i 's are integers such that

$$0 \leq d_i < b, \quad i = 0, \dots, p-1$$

and E is an integer such that

$$L \leq E \leq U$$

Floating-Point Arithmetic

Examples:

$$x = \pm b^E \cdot \sum_{i=0}^{p-1} \frac{d_i}{b^i}$$

Decimal ($p = 4$):

$$123.4_{10} = 10^2 \cdot \left(\frac{1}{10^0} + \frac{2}{10^1} + \frac{3}{10^2} + \frac{4}{10^3} \right) = (1234, 2)$$

Decimal ($p = 6$):

$$123.400_{10} = 10^2 \cdot \left(\frac{1}{10^0} + \frac{2}{10^1} + \frac{3}{10^2} + \frac{4}{10^3} + \frac{0}{10^4} + \frac{0}{10^5} \right) = (123400, 2)$$

Obviously, 123.4 and 123.400 represent the same value, but the 6-digit system can represent numbers that the 4-digit system cannot: e.g., 123.41

Floating-Point Arithmetic

Examples:

$$x = \pm b^E \cdot \sum_{i=0}^{p-1} \frac{d_i}{b^i}$$

Binary ($p = 5$):

$$100.11_2 = 2^2 \cdot \left(\frac{1}{2^0} + \frac{0}{2^1} + \frac{0}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} \right) = (10011, 2)$$

Binary ($p = 8$):

$$100.1100_2 = 2^2 \cdot \left(\frac{1}{2^0} + \frac{0}{2^1} + \frac{0}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{0}{2^5} + \frac{0}{2^6} + \frac{0}{2^7} \right) = (1001100, 2)$$

Again, these are the same values, but the second system can represent more numbers

Floating-Point Arithmetic

Properties of floating-point systems:

Special numbers:

- Total numbers possible:
$$2(b-1)b^{p-1}(U-L+1)+1$$
- **Underflow limit** (UFL): The smallest possible number that can be represented by a floating point system:
$$\text{UFL} = b^L$$
- **Overflow limit** (OFL): The largest possible number that can be represented by a floating point system:
$$\text{OFL} = b^{U+1}(1 - b^{-p})$$
- **Machine precision** (ϵ_{mach}): :
The smallest possible relative error between a real number and its float representation, or, the smallest possible relative difference between two floats: $\epsilon_{mach} = b^{1-p}, \frac{1}{2}b^{1-p}$ (depends on how the computer rounds numbers)

Floating-Point Arithmetic

Example 1.9 Floating-Point System. An example floating-point system is illustrated in Fig. 1.3, where the tick marks indicate all of the 25 floating-point numbers in a system having $\beta = 2$, $p = 3$, $L = -1$, and $U = 1$. For this system, the largest number is $\text{OFL} = (1.11)_2 \times 2^1 = (3.5)_{10}$, and the smallest positive normalized number is $\text{UFL} = (1.00)_2 \times 2^{-1} = (0.5)_{10}$. This is a very tiny, toy system for illustrative purposes only, but it is in fact characteristic of floating-point systems in general: at a sufficiently high level of magnification, every normalized floating-point system looks essentially like this one—grainy and unequally spaced.

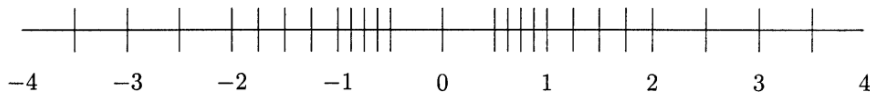


Figure: Example floating-point system (Heath)

Floating-Point Arithmetic

Try to construct 7 in binary:

- Left to right: 8's place, 4's place, 2's place, 1's place
- Put a 1 in the largest digit's that isn't larger than 7 (i.e., 4)
- 3 remainder
- Put a 1 in the largest digit's that isn't larger than 3 (i.e., 2)
- 1 remainder
- Put a 1 in the largest digit's that isn't larger than 1 (i.e., 1)
- Done: $7_{10} = 0111_2$

Binary	Decimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

Floating-Point Arithmetic

Example:

Finally, let's try to construct 0.1 in binary

```
x = 0.1
```

```
n = 20
```

```
total = 0.0
```

```
binary = '0.'
```

```
for i in range(1,n):
```

```
    test = total + 2**(-i)
```

```
    if test < x:
```

```
        total = test
```

```
        binary += '1'
```

```
    else:
```

```
        binary += '0'
```

```
    # end
```

```
# end
```

```
print(x, total, binary)
```

- Left to right: 1's place, 1/2's place, 1/4's place, etc.
- Put a 1 in the largest digit's that isn't larger than 0.1 (or its remainder at later steps)

Conclusion:

0.1 is an infinitely repeating binary expansion (much like $1/3$ is in decimal form)

The Newton-Raphson Method

The Newton-Raphson Method

Defn: Newton-Raphson:

- Moving from brute force to bisection was an improvement because we were able to incorporate additional knowledge about the search space: that it was sorted/ordered
- We can continue to make more efficient methods by utilizing more information about the search space

The **Newton-Raphson method** is an algorithm specifically designed to find roots of real-valued functions which utilizes the derivative of the function

Starting with some initial guess at a solution x_0 , the method iteratively improves the guess via the following formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (1)$$

The Newton-Raphson Method

The Newton-Raphson Method:

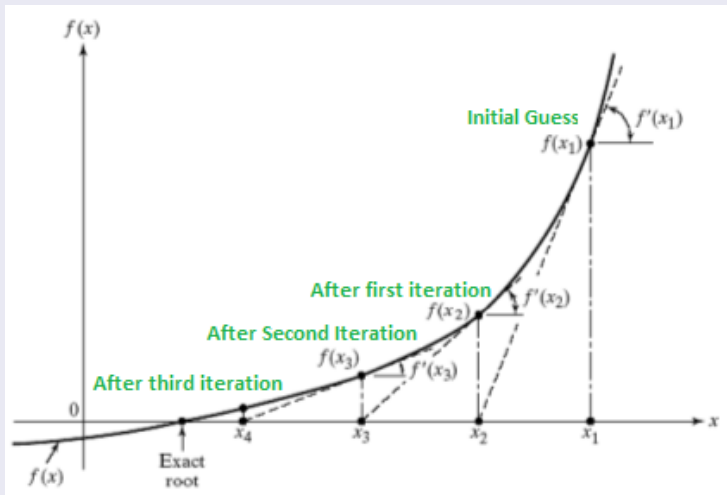


Figure: Visual depiction of the algorithm

The Newton-Raphson Method

Deriving Newton-Raphson:

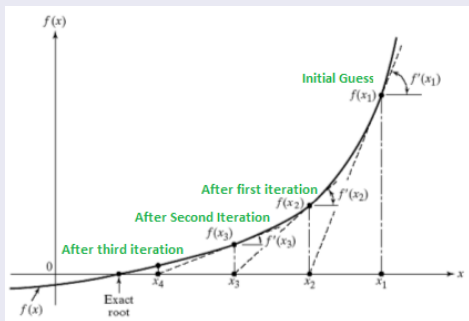
Find the equation of the line from $f(x_1)$ to 0:

$$y - \hat{y} = m(x - \hat{x})$$

$$y - f(x_1) = f'(x_1)(x - x_1)$$

$$0 - f(x_1) = f'(x_1)(x_2 - x_1)$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}$$



The Newton-Raphson Method

The Newton-Raphson Method:

Solve $f(x) = x^2 - 2 = 0$ (i.e., find $\sqrt{2}$)

```
c = 2
eps = 0.01
ans = 10

while abs(ans**2 - c) >= eps:
    ans = ans - (ans**2 - c)/(2*ans)
# end

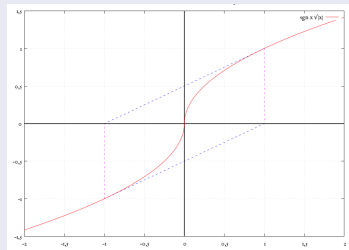
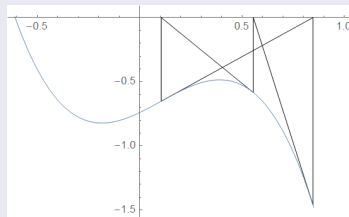
print('sqrt(', c, ') is about', ans)
```

- $f'(x) = 2x$
- The closer the initial guess is to the answer, the fewer steps required
- This function has multiple roots

The Newton-Raphson Method

Limitations of Newton-Raphson:

- Method can get stuck in a loop on certain functions
- If the method ever lands on $f'(x_i) = 0$, then the method crashes due to a divide by zero error



The Newton-Raphson Method

3 methods compared:

Brute force (exhaustive elimination):

- Simple
- Most real-world cases have a search space way too large

Bisection:

- Far more efficient than brute force
- Needs search space to be ordered (many are, not a big deal)
- `high, low` values need to contain the answer between them

Newton-Raphson:

- Most efficient of all three
- Needs search space to be ordered (many are, not a big deal)
- Needs to evaluate derivative (in many real-world applications, this is non-trivial and computationally expensive)
- Method behaves poorly on certain functions